

Course Project:
A Secure Peer-to-Peer Messaging System

Presented by:
Philippe Laporte

Foundations of Cryptography 6110
Information Systems Engineering
Concordia University

April 14 2025

Abstract

We present a secure peer-to-peer (P2P) messaging system with a command-line interface (CLI) that ensures confidentiality, integrity, and authentication of messages. The system will utilize hybrid asymmetric encryption, where RSA is used for key exchange and AES 128 in CBC mode for actual communication. RSA digital signatures and HMAC authentication will be used to provide message integrity and authenticity. We deliver Perfect Forward Secrecy of the RSA keys as an alternative to Certificate-based Authentication.

1. Introduction

1.1 Problem Statement

With the growing need for secure digital communication, ensuring the confidentiality, integrity, and authenticity of messages is critical. Traditional messaging platforms often rely on centralized servers, which introduce risks such as data breaches, unauthorized access, and surveillance. A peer-to-peer (P2P) messaging system eliminates these risks by removing centralized control and enabling direct, secure communication between users.

This project delivers a secure P2P messaging system that leverages hybrid cryptographic techniques to protect messages from eavesdropping, tampering, and impersonation attacks. The system will provide end-to-end security, ensuring that messages remain private and unaltered.

1.2 Project Objectives

The primary goal of this project is to design and implement a secure messaging system that:

- Provides an ephemeral peer-to-peer communication model that does not use X.509 for RSA key certification and instead delivers Perfect Forward Secrecy, ensuring that even if a session establishment key is compromised, past and future communications remain secure
- Implements a command-line interface (CLI) for direct user interaction.
- Uses asymmetric RSA for key exchange to securely distribute symmetric keys.
- Uses AES-CBC 128 bits encryption to ensure message confidentiality.
- Implements RSA digital signatures to authenticate peers during session establishment and HMAC to verify message integrity and authenticity during the established message exchange session
- Implement proper encryption padding so as not to compromise the ciphertext, and by the same token making RSA encryption non-deterministic

2. System Design

2.1 Overview of the Secure Messaging System

The system delivers a secure peer-to-peer communication by exchanging encrypted messages over a TCP/IP socket connection. The encryption process involves:

- Generating RSA key pairs for each peer.
- Exchanging public keys
- Using RSA to securely exchange AES session keys (encryption and signing) using OAEP padding for encryption and PSS padding for the signature
- RSA signatures are applied to a SHA-256 Hash of the plaintext (Hash-then-Sign), the AES key
- The AES key pair is small enough not to require digesting it before encryption
- Encrypting all message content using AES in CBC mode

2.2 Cryptographic Techniques Used

- **Key Exchange:** RSA for securely transmitting AES session keys.
- **Message Encryption:** AES for encrypting message contents.
- **Authentication:** Digital signatures with RSA and HMAC-SHA-256 with AES.
- **Hashing:** SHA-256
- **Padding:** Improper padding can compromise Encryption. We will use appropriate padding for both encryption and signing operations

2.3 Key Exchange Process

- Each peer generates an RSA key pair at the start of the session.
- The AES session and signing keys are encrypted using the recipient's RSA public key, signed with the sender's RSA private key, and appended to the encrypted keys.
- OAEP padding is used for encryption and PSS padding is used for Signing, both paddings being powered by SHA256 Hashes
- Upon reception, the recipient decrypts the AES key and verifies the digital signature using the sender's RSA public key.

2.4 Message Transmission

- Once the secure session is established, all communication is encrypted with AES-CBC.
- Each AES-encrypted message appends an HMAC signature for authentication.
- The system uses PKCS7 padding, initialization vectors (IV), and a secure encryption mode (AES-CBC).

2.5 Communication Protocol Flow

participant "Initiator" as I, the session initiator peer

participant "Responder" as R, the peer listening for incoming session requests

I -> R: Send RSA public key

I <- R: Send RSA public key

I -> R: Send AES keys, for encryption and signing, encrypted with R's RSA public key) appended with the RSA-signed hash

R: Decrypt AES key and verify signature using I's public key

I <- R: Send Message of the Day (MOTD) un-encrypted.

loop Secure Messaging

I -> R: Send AES-CBC-encrypted message + HMAC

I <- R: Send AES-CBC-encrypted reply + HMAC

When the conversation ends, the responder awaits the next connection from an initiator

3.Implementation

3.1 Technology Stack

- **Programming Language:** Python
- **Cryptographic Library:** The `cryptography` module. This module depends on the OpenSSL C library for all cryptographic operations. OpenSSL is the de facto standard for cryptographic libraries and provides high performance along with various certifications that may be relevant to developers. This C library is interfaced through the cPython Runtime's C API.
- **Networking:** Plain TCP/IP sockets are used for peer-to-peer communication

3.2 CLI-based User Interface

- Users select their role (Initiator or Responder).
- Specify host/port details for connection, with appropriate defaults
- Peer messages are UTF-8 encoded strings sent as bytes
- MOTD (Message-of-The-Day) is sent unencrypted by responder so as not to enable a known-plaintext attack

3.3 Object-Oriented Design

- Implement reusable classes for encryption, authentication, and network communication.
- Ensure modularity for easy extension.

3.4 AES Message Exchange

- For the AES-encrypted message exchange, we use the `Fernet` class from the `cryptography` module, which includes timing data to help prevent replay attacks.
- It uses PKCS7 padding, and an Encrypt-then-MAC strategy with a new IV for each encryption and HMAC with SHA-256 for signing/authentication
- The encryption key is 16 bytes and is generated using the widely accepted `os.urandom` function, and so is the signing key
- base64 encoding used to transmit in text format for media like email

3.5 Source code

The Initiator and Responder classes share a `SecurePeer` super class which knows how to generate an RSA key pair. A thorough protocol flow narrative appears in comments and in the print verbose as a messaging session happens:

```
# securepeer.py
#
# RSA+AES Hybrid Secure TCP channel
#
# Author: Philippe Laporte <philippe.laporte@mail.concordia.ca>
#
# Winter 2025 - Foundations of Cryptography - INSE 6110
#
# In a Java-sense, all class fields are considered private, some may also be static, and
all class methods are considered public
#
# Run python securepeer.py -h for usage, and pip install cryptography for dependencies

import sys
import socket
import argparse

from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.fernet import Fernet
from cryptography.exceptions import InvalidSignature
```

```

#-----
# SecurePeer abstract base class
# -----

class SecurePeer:
    """Abstract role for Secure Peer-to-Peer chat"""

    RSA_PUBLIC_EXPONENT = 65537
    RSA_KEY_SIZE = 4096          # bits

    STRING_ENCODING = 'utf-8'

    MAX_CHAT_MESSAGE_SIZE = 10000 # bytes

    SOCKET_TIMEOUT_ESTABLISH_SESSION = 10 # seconds

    def __init__(self, host, port):
        self.host = host
        self.port = port

        #generate key pair
        print('Generating RSA key pair with exponent', self.RSA_PUBLIC_EXPONENT, 'and key
size', self.RSA_KEY_SIZE, 'bits')

        self.private_key = rsa.generate_private_key(
            public_exponent=self.RSA_PUBLIC_EXPONENT,
            key_size=self.RSA_KEY_SIZE
        )

        self.public_key_pem = self.private_key.public_key().public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        )

#-----
# Initiator
# -----

class Initiator(SecurePeer):
    """Initiator role for Secure Peer-to-Peer chat"""

    def connect_to_peer(self):
        print('Initiating conversation')

        self.client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        self.client_socket.settimeout(self.SOCKET_TIMEOUT_ESTABLISH_SESSION)

        print('Connecting to host', self.host, 'on port', self.port)

        self.client_socket.connect((self.host, self.port))

        print('Connected')

    def establish_session(self):
        print('Sending public key')

        self.client_socket.sendall(self.public_key_pem)

        print('Awaiting peer public key')

        peer_public_key_pem = self.client_socket.recv(self.RSA_KEY_SIZE) # Key size is
upper-bound on pem size

```

```

print('Received peer public key')

peer_public_key = serialization.load_pem_public_key(
    peer_public_key_pem
)

print('Generating AES key')

aes_key = Fernet.generate_key()

print('Encrypting AES key with peer\'s public key')

aes_key_cipher_text = peer_public_key.encrypt(
    aes_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print('Generating Signature from AES key with private key')

signature = self.private_key.sign(
    aes_key,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print('Sending AES key with appended Signature')

self.client_socket.sendall(aes_key_cipher_text + signature)

print('Initiating AES session')

self.fernet = Fernet(aes_key)

# relax the timeouts to allow for infinite reply delays
self.client_socket.settimeout(None)

def have_conversation(self, next_message, on_peer_message):
    print('Awaiting MOTD\n')

    # motd is sent plainly
    motd_text = self.client_socket.recv(self.MAX_CHAT_MESSAGE_SIZE)
    on_peer_message(str(motd_text, self.STRING_ENCODING))
    message = next_message()

    while(message): # empty message ends conversation
        # send
        cipherMessage = self.fernet.encrypt(bytes(message, self.STRING_ENCODING))
        self.client_socket.sendall(cipherMessage)

        # receive
        cipherReply = self.client_socket.recv(self.MAX_CHAT_MESSAGE_SIZE)
        reply = self.fernet.decrypt(cipherReply)

        # display message and get next one
        on_peer_message(str(reply, self.STRING_ENCODING))
        message = next_message()

def shutdown(self):
    try:
        self.client_socket.close()

```

```

except AttributeError:
    pass

#-----
# Responder
#-----

class Responder(SecurePeer):
    """Responder role for Secure Peer-to-Peer chat"""

    MOTD = 'Welcome to our secure chat!'

    def listen_for_peer_connection(self):
        print('Listening for incoming connections on port', self.port)

        self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.server_socket.bind((self.host, self.port))
        self.server_socket.listen()

    def await_peer_connection(self):
        print('Waiting for incoming conversation')

        self.client_socket, client_address = self.server_socket.accept()

        print('Connected to initiator peer from host', client_address[0])

        self.client_socket.settimeout(self.SOCKET_TIMEOUT_ESTABLISH_SESSION)

    def establish_session(self):
        try:
            peer_public_key_pem = self.client_socket.recv(self.RSA_KEY_SIZE)    # Key size is
upper-bound on pem size

            peer_public_key = serialization.load_pem_public_key(
                peer_public_key_pem
            )

            print('Received peer public key')

            print('Sending public key')

            self.client_socket.sendall(self.public_key_pem)

            print('Waiting for AES key')

            aes_key_cipher_payload = self.client_socket.recv(self.RSA_KEY_SIZE // 4)

            cipher_payload_part_size = self.RSA_KEY_SIZE // 4 // 2

            aes_key_cipher_text = aes_key_cipher_payload[:cipher_payload_part_size]
            signature = aes_key_cipher_payload[cipher_payload_part_size:]

            print('Received AES key and signature')

            print('Decrypting AES key using private key')

            aes_key = self.private_key.decrypt(
                aes_key_cipher_text,
                padding.OAEP(
                    mgf=padding.MGF1(algorithm=hashes.SHA256()),
                    algorithm=hashes.SHA256(),
                    label=None

```



```

    )
)

print('Verifying signature using peer public key')

peer_public_key.verify(
    signature,
    aes_key,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

self.fernet = Fernet(aes_key)

# relax the timeouts to allow for infinite reply delays
self.client_socket.settimeout(None)

except InvalidSignature as e:
    print ("Invalid signature for AES key message")
    raise e
except Exception as e:
    print(e)
    raise e


def have_conversation(self, next_message, on_peer_message):
    print('Sending MOTD\n')

    # send it plainly so as not to compromise the encryption with a known plaintext attack
    message = self.MOTD
    self.client_socket.sendall(bytes(message, self.STRING_ENCODING))

    while(message): # empty message ends conversation
        # receive
        cipherReply = self.client_socket.recv(self.MAX_CHAT_MESSAGE_SIZE)
        reply = self.fernet.decrypt(cipherReply)

        # display message and get next one
        on_peer_message(str(reply, self.STRING_ENCODING))
        message = next_message()

        # send
        cipherMessage = self.fernet.encrypt(bytes(message, self.STRING_ENCODING))
        self.client_socket.sendall(cipherMessage)


def shutdown_client(self):
    try:
        self.client_socket.close()
    except AttributeError:
        pass


def shutdown(self):
    print('Shutting down')

    self.shutdown_client()
    self.server_socket.close()


#-----
# Main drivers
#-----

```

```

MESSAGE_PROMPT = '-> '

def initiator(host, port):
    try:
        peer = Initiator(host, port)
        peer.connect_to_peer()
        peer.establish_session()

        def next_message():
            return input(MESSAGE_PROMPT)
        def on_peer_message(message):
            print(message)

        peer.have_conversation(next_message, on_peer_message)
    except KeyboardInterrupt:
        sys.stderr.write('\r') # eat the ^C TODO not working in Powershell!
    except Exception as e:
        print(e)
    finally:
        print('Ending conversation')
        try:
            peer.shutdown()
        except UnboundLocalError:
            pass

def responder(port):
    try:
        peer = Responder(socket.gethostbyname(socket.gethostname()), port)
        peer.listen_for_peer_connection()

        def next_message():
            return input(MESSAGE_PROMPT)
        def on_peer_message(message):
            print(message)

        while (True):
            try:
                peer.await_peer_connection()
                peer.establish_session()
                peer.have_conversation(next_message, on_peer_message)
            except Exception as e:
                print('Ending conversation')
            finally:
                peer.shutdown_client()
    except KeyboardInterrupt:
        sys.stderr.write('\r') # eat the ^C TODO not working in Powershell!
    except Exception as e:
        print(e)
    finally:
        try:
            peer.shutdown()
        except UnboundLocalError:
            pass

#-----
# Main
#-----

DEFAULT_CHAT_PORT = 7676

if __name__ == "__main__":
    # parse command-line arguments
    parser = argparse.ArgumentParser()
    parser.add_argument("role", help="specifies which role the peer is to play, either
'initiator' or 'responder'")

```

```

    parser.add_argument("--host", type=str, help="specifies which host the initiator is to
connect to")
    parser.add_argument("--port", type=int, help="specifies which port the initiator is to
connect to or the responder is to listen on. Default is " + str(DEFAULT_CHAT_PORT))
    args = parser.parse_args()

    if (args.role == 'initiator'):
        initiator(socket.gethostbyname(socket.gethostbyname()) if args.host is None else
args.host, DEFAULT_CHAT_PORT if args.port is None else args.port)
    elif (args.role == 'responder'):
        # validate host
        if (args.host != None):
            print('Host ignored for responder role', file=sys.stderr)
            responder(DEFAULT_CHAT_PORT if args.port is None else args.port)
        else:
            print('Unknown role, exiting', file=sys.stderr)

```

Here we have the code for the Fernet module:

```

# This file is dual licensed under the terms of the Apache License, Version
# 2.0, and the BSD License. See the LICENSE file in the root of this repository
# for complete details.

from __future__ import annotations

import base64
import binascii
import os
import time
import typing
from collections.abc import Iterable

from cryptography import utils
from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives import hashes, padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.hmac import HMAC

class InvalidToken(Exception):
    pass

_MAX_CLOCK_SKEW = 60

class Fernet:
    def __init__(
        self,
        key: bytes | str,
        backend: typing.Any = None,
    ) -> None:
        try:
            key = base64.urlsafe_b64decode(key)
        except binascii.Error as exc:
            raise ValueError(
                "Fernet key must be 32 url-safe base64-encoded bytes."
            ) from exc
        if len(key) != 32:
            raise ValueError(
                "Fernet key must be 32 url-safe base64-encoded bytes."
            )

        self._signing_key = key[:16]
        self._encryption_key = key[16:]

```

```

@classmethod
def generate_key(cls) -> bytes:
    return base64.urlsafe_b64encode(os.urandom(32))

def encrypt(self, data: bytes) -> bytes:
    return self.encrypt_at_time(data, int(time.time()))

def encrypt_at_time(self, data: bytes, current_time: int) -> bytes:
    iv = os.urandom(16)
    return self._encrypt_from_parts(data, current_time, iv)

def _encrypt_from_parts(
    self, data: bytes, current_time: int, iv: bytes
) -> bytes:
    utils._check_bytes("data", data)

    padder = padding.PKCS7(algorithms.AES.block_size).padder()
    padded_data = padder.update(data) + padder.finalize()
    encryptor = Cipher(
        algorithms.AES(self._encryption_key),
        modes.CBC(iv),
    ).encryptor()
    ciphertext = encryptor.update(padded_data) + encryptor.finalize()

    basic_parts = (
        b"\x80"
        + current_time.to_bytes(length=8, byteorder="big")
        + iv
        + ciphertext
    )

    h = HMAC(self._signing_key, hashes.SHA256())
    h.update(basic_parts)
    hmac = h.finalize()
    return base64.urlsafe_b64encode(basic_parts + hmac)

def decrypt(self, token: bytes | str, ttl: int | None = None) -> bytes:
    timestamp, data = Fernet._get_unverified_token_data(token)
    if ttl is None:
        time_info = None
    else:
        time_info = (ttl, int(time.time()))
    return self._decrypt_data(data, timestamp, time_info)

def decrypt_at_time(
    self, token: bytes | str, ttl: int, current_time: int
) -> bytes:
    if ttl is None:
        raise ValueError(
            "decrypt_at_time() can only be used with a non-None ttl"
        )
    timestamp, data = Fernet._get_unverified_token_data(token)
    return self._decrypt_data(data, timestamp, (ttl, current_time))

def extract_timestamp(self, token: bytes | str) -> int:
    timestamp, data = Fernet._get_unverified_token_data(token)
    # Verify the token was not tampered with.
    self._verify_signature(data)
    return timestamp

@staticmethod
def _get_unverified_token_data(token: bytes | str) -> tuple[int, bytes]:
    if not isinstance(token, (str, bytes)):
        raise TypeError("token must be bytes or str")

    try:
        data = base64.urlsafe_b64decode(token)
    except (TypeError, binascii.Error):
        raise InvalidToken

```

```

        if not data or data[0] != 0x80:
            raise InvalidToken

        if len(data) < 9:
            raise InvalidToken

        timestamp = int.from_bytes(data[1:9], byteorder="big")
        return timestamp, data

    def _verify_signature(self, data: bytes) -> None:
        h = HMAC(self._signing_key, hashes.SHA256())
        h.update(data[:-32])
        try:
            h.verify(data[-32:])
        except InvalidSignature:
            raise InvalidToken

    def _decrypt_data(
        self,
        data: bytes,
        timestamp: int,
        time_info: tuple[int, int] | None,
    ) -> bytes:
        if time_info is not None:
            ttl, current_time = time_info
            if timestamp + ttl < current_time:
                raise InvalidToken

            if current_time + _MAX_CLOCK_SKEW < timestamp:
                raise InvalidToken

        self._verify_signature(data)

        iv = data[9:25]
        ciphertext = data[25:-32]
        decryptor = Cipher(
            algorithms.AES(self._encryption_key), modes.CBC(iv)
        ).decryptor()
        plaintext_padded = decryptor.update(ciphertext)
        try:
            plaintext_padded += decryptor.finalize()
        except ValueError:
            raise InvalidToken
        unpadder = padding.PKCS7(algorithms.AES.block_size).unpadder()

        unpadded = unpadder.update(plaintext_padded)
        try:
            unpadded += unpadder.finalize()
        except ValueError:
            raise InvalidToken
        return unpadded

```

4. Security Analysis and Discussion

4.1. Potential Vulnerabilities & Risks

4.1.1 Key Exchange Risks

- X.509 certificates are not used and properly validated, a man-in-the-middle (MITM) attack could replace the RSA public key if they can somehow locate a communication session as it happens. We sure don't want to be communicating with an impersonator.
- Generating RSA key pairs dynamically (on-the-fly) is useful in this scenario where keys are needed temporarily and for specific sessions. Randomizing the communication ports will further enhance unpredictability
- Then we can deliver Perfect Forward Secrecy which is a formidable result
- Diffie-Hellman should be researched as the next increment in providing secrecy, replacing the RSA key exchange

4.1.2 Timing Attacks

- RSA decryption and signature verification may be susceptible to timing attacks due to how Python handles integer operations.
- This will apply to the pure Python implementation project `python-rsa`, but not to the cryptography module we use as it is a wrapper over OpenSSL.

4.1.3 Replay Attacks

- An attacker could capture and resend messages, even if they cannot decrypt them
- From the RSA viewpoint, we provide ephemeral keys
- As for AES, the Fernet library provides timing data to mitigate this risk as well as a new IV for each message encryption

4.1.4 HMAC & Message Integrity

- If HMAC is not properly verified, an attacker might modify messages without detection.
- The Fernet library does provide such verification

4.1.5. AES-CBC Mode Considerations

- Predictable IVs could allow certain cryptographic attacks.
- Improper padding handling may lead to padding oracle vulnerabilities.

4.2 Risks reported by the AI Code Analysis Agent (Replit)

Once we were done with the coding we used an AI analysis tool and prompted it to find weaknesses in the program.

4.2.1 With prompt “find weaknesses in this code”, the initial report had several comments. I list here only the ones related to encryption

Tool output:

I've examined the code and found some security weaknesses in this secure chat implementation. Here are the key issues:

Hardcoded MOTD - The Message of the Day is hardcoded and unchangeable.

No Certificate Validation - The public key exchange lacks certificate validation, making it vulnerable to man-in-the-middle attacks.

Missing IV for AES - Using Fernet for encryption, but not explicitly managing IVs or checking for replay attacks.

RSA Key Reuse - Keys are generated each time with no persistence, risking repeated identical key generation.

Missing IV Management - While using Fernet for encryption, there's no explicit management of initialization vectors or protection against replay attacks.

The most critical issues are the lack of certificate validation (enabling MITM attacks)

4.2.2 It would indeed be best to rotate the MOTD to make it harder to identify which communication protocol the program is using

This version of the code is indeed not using certificates, so it identified this issue very clearly.

As previously stated we believe the ad-hoc nature of the communication system mitigates MITM attacks.

The missing IV report is wrong/incorrect. This is feedback to report on the quality of the Tool's analysis

It correctly identifies that the system risks repeating RSA keys as they are ephemeral.

Unrelated to encryption we were told to put timeouts on sockets so as to mitigate against DOS attacks.

Conclusion

We have provided a Secure Real Time Peer-to-Peer Messaging System using robust cryptographic techniques. By leveraging RSA for key exchange and AES for encryption, the system ensures confidentiality, integrity, and the targeted level of authentication.

References

- [Python cryptography module](#)
- [cryptography module github](#)
- [Fernet Module specifications](#)
- [Fernet class in github](#)
- [Pure Python RSA implementation](#)
- [Research on timing attacks and constant-time comparison in Python](#)
- [OpenSSL library](#)